Luniero Marketing Agent — Slash Command Interface

Design Philosophy

Users shouldn't learn a new interface. They already know: - `/command` syntax from Slack, Discord, Telegram - Natural language descriptions - Conversational follow-ups

Principle: `/<action> <natural language description>`

Command Categories

1. Content Creation

```
/write <what you want>
```

Examples:

```
/write a LinkedIn post about AI trends for 2026
/write a Twitter thread on why startups should use AI automation
/write a blog post about content marketing best practices
/write Instagram captions for our new product launch (5 variations)
/write email sequence for new client onboarding (3 emails)
/write ad copy for Facebook campaign targeting small business owners
```

What happens: 1. Agent parses intent (platform, content type, topic) 2. Loads client context (if client specified or default) 3. Runs: Brief → Draft → Polish → Review pipeline 4. Returns content + asks for approval

Flags:

```
/write ... --client acme           # Specify client
/write ... --platform linkedin     # Force platform
/write ... --tone casual           # Override tone
/write ... --length short          # Adjust length
/write ... --variations 3          # Multiple versions
```

---

2. Quick Content (Skip Full Pipeline)

```
/quick <what you want>
```

Examples:

```
/quick LinkedIn post about our new feature
/quick tweet celebrating 1000 followers
/quick Instagram story caption for team photo
```

What happens: - Skips full pipeline - Single LLM call with client context - Faster but less refined - Good for simple, time-sensitive content

---

3. Content Calendar

```
/calendar <timeframe> [platform]
```

Examples:

```
/calendar this week
/calendar next week linkedin
/calendar february instagram
/calendar q1 all
```

What happens: 1. Loads client content pillars 2. Researches trending topics 3. Generates content calendar 4. Returns schedule with post ideas

Output:

```
 Week of Feb 10-14 — Acme Corp LinkedIn

Mon 10: Thought leadership — "Why async beats meetings"
Tue 11: Product tip — Hidden feature spotlight
Wed 12: Industry news — React to AI announcement
Thu 13: Customer story — Quote from happy client
Fri 14: Weekend thought — Reflection post

Approve this calendar? (yes/edit/regenerate)
```

---

4. Reports

```
/report <type> [timeframe]
```

Examples:

```
/report weekly
/report monthly
/report campaign "Summer Launch"
/report performance last 30 days
/report competitor analysis
```

What happens: 1. Pulls analytics from connected platforms 2. Compiles metrics 3. Generates insights and recommendations 4. Outputs formatted report (text + PDF option)

Output:

```
 Acme Corp — Weekly Report (Feb 3-9)

HIGHLIGHTS
 LinkedIn engagement up 23%
 Best post: "AI trends" (2.4K impressions)
 Twitter reach down 8%

TOP CONTENT
1. "Why we ditched spreadsheets" — 847 engagements
2. Product demo video — 412 views
3. Team spotlight — 234 likes

RECOMMENDATIONS
→ Double down on LinkedIn carousels
→ Test Twitter threads instead of links
→ Post customer stories on Wednesdays
```

```
Want the full PDF? (/report pdf)
```

---

5. Research

```
/research <topic or request>
```

Examples:

```
/research trending topics in B2B SaaS
/research what competitors are posting this week
/research best hashtags for project management content
/research viral posts in our industry
/research audience sentiment about AI tools
```

What happens: 1. Research Agent activates 2. Searches web, social platforms, news 3. Compiles findings 4. Returns actionable insights

---

6. Client Management

```
/client <action> [details]
```

Examples:

```
/client list                              # Show all clients
/client switch acme                       # Change active client
/client new "Initech Corp" B2B SaaS       # Create new client
/client voice                             # Show current client's
brand voice
/client voice update "more casual"        # Update brand voice
/client pillars                           # Show content pillars
/client pillars add "sustainability"      # Add content pillar
```

Active Client: - One client is “active” at a time - All commands default to active client - Switch with `/client switch <name>` - Or specify per-command with `--client`

---

7. Approvals & Revisions

```
/approve [job_id]               # Approve pending content
/revise <feedback>              # Request changes
/reject [reason]                # Reject and explain why
```

Examples:

```
/approve                        # Approve last pending item
/approve job_abc123             # Approve specific job
/revise make it more casual and add a statistic
/revise stronger hook, remove the question at the end
/reject off-brand, try again with different angle
```

What happens: - `/approve` → Content moves to next stage or completion - `/revise` → Sends feedback to Draft/Polish agent, loops back - `/reject` → Logs feedback, starts fresh

---

8. Schedule & Publish

```
/schedule <when>                # Schedule approved content
/publish now                    # Publish immediately
/queue                          # Show scheduled posts
```

Examples:

```
/schedule tomorrow 9am
/schedule monday 10am EST
/schedule optimal               # Let AI pick best time
/publish now
/queue                          # See what's scheduled
/queue cancel job_abc123        # Cancel scheduled post
```

---

9. History & Status

```
/status [job_id]                # Check job status
/history [count]                # Recent jobs
/show <job_id>                  # Show full output
```

Examples:

```
/status                         # Status of last job
/status job_abc123              # Specific job
/history                        # Last 10 jobs
/history 20                     # Last 20 jobs
/show job_abc123                # Full content of a job
```

---

10. Help & Settings

```
/help [command]                 # Get help
/settings                       # Show current settings
/settings <key> <value>         # Update setting
```

Examples:

```
/help                           # All commands
/help write                     # Help for /write
/settings                       # Show all settings
/settings default_platform linkedin
/settings auto_approve false
/settings timezone America/Chicago
```

Conversational Mode

After any command, users can follow up naturally:

```
User: /write LinkedIn post about AI trends

Agent: Here's your post:
[content]

Approve? (yes/edit/regenerate)

User: make it shorter and punchier

Agent: Here's the revised version:
```

```
[shorter content]

User: perfect, schedule it for monday

Agent: Scheduled for Monday Feb 10, 9:00 AM. 
```

The agent understands context and follow-ups.

Command Aliases

Multiple ways to say the same thing:

Primary	Aliases
`/write`	`/create`, `/draft`, `/make`
`/quick`	`/q`, `/fast`
`/calendar`	`/cal`, `/plan`
`/report`	`/analytics`, `/stats`
`/research`	`/find`, `/search`
`/client`	`/c`
`/approve`	`/ok`, `/yes`, `/lgtm`
`/revise`	`/edit`, `/change`, `/fix`
`/schedule`	`/sched`, `/post`
`/status`	`/s`, `/?`
`/help`	`/h`, `/?`

Smart Parsing

The agent intelligently parses natural language:

```
/write linkedin post about our new feature
```

Parsed as:

```
{
  "command": "write",
  "platform": "linkedin",
  "content_type": "post",
  "topic": "our new feature",
  "client": "[active_client]"
}
```

```
/write 5 tweet variations about the product launch with casual tone
```

Parsed as:

```
{
  "command": "write",
  "platform": "twitter",
  "content_type": "tweet",
  "topic": "product launch",
  "variations": 5,
  "tone": "casual"
}
```

Interactive Prompts

When info is missing, agent asks:

```
User: /write a blog post

Agent: I'll create a blog post for you.

What topic should I cover?

User: AI automation for small businesses

Agent: Got it. Any specific angle or key points to include?

User: focus on cost savings and time efficiency

Agent: Creating your blog post now...
[runs pipeline]
```

Response Format

All responses follow a consistent format:

Content Output

```
 **LinkedIn Post** — Acme Corp
━━━━━━━━━━━━━━━━━━━━━━━━━━━━

[content here]

━━━━━━━━━━━━━━━━━━━━━━━━━━━━
 Words: 127 | Score: 94/100
 #AIautomation #productivity

 Approve    ✏ Revise    Regenerate
```

Status Update

```
 Job: job_abc123
 Status: polishing
 Client: Acme Corp
 Type: LinkedIn post
 Started: 2 min ago
```

Error

```
 Error: Could not connect to LinkedIn API

Possible fixes:
1. Check API credentials in /settings
2. Try again in a few minutes
3. Use /help linkedin for setup guide
```

Implementation Architecture

Parser Module (src/cli/parser.ts)

```
interface ParsedCommand {
  command: string;
  subcommand?: string;
  topic?: string;
  platform?: string;
  contentType?: string;
  client?: string;
  flags: Record<string, any>;
  raw: string;
}

function parseCommand(input: string): ParsedCommand {
  // Handle slash commands
  if (input.startsWith('/')) {
    return parseSlashCommand(input);
  }

  // Handle conversational follow-ups
  return parseConversational(input);
}

function parseSlashCommand(input: string): ParsedCommand {
  const [command, ...rest] = input.slice(1).split(' ');
  const content = rest.join(' ');

  // Extract flags (--key value)
  const flags = extractFlags(content);
  const cleanContent = removeFlags(content);

  // Smart parsing based on command
  switch (command) {
    case 'write':
    case 'create':
    case 'draft':
      return parseWriteCommand(cleanContent, flags);
    case 'calendar':
    case 'cal':
      return parseCalendarCommand(cleanContent, flags);
    // ... etc
  }
}

function parseWriteCommand(content: string, flags: Record<string,
any>): ParsedCommand {
  // Use NLP or pattern matching to extract:
  // - Platform (linkedin, twitter, instagram, etc.)
  // - Content type (post, thread, carousel, story, etc.)
  // - Topic
  // - Modifiers (casual, short, variations, etc.)

  const platform = extractPlatform(content) || flags.platform;
  const contentType = extractContentType(content) || 'post';
  const topic = extractTopic(content);

  return {
```

```
        command: 'write',
        platform,
        contentType,
        topic,
        flags,
        raw: content,
      };
    }
```

Command Router (src/cli/router.ts)

```
    async function routeCommand(parsed: ParsedCommand, context:
SessionContext) {
      switch (parsed.command) {
        case 'write':
          return await handleWriteCommand(parsed, context);

        case 'quick':
          return await handleQuickCommand(parsed, context);

        case 'calendar':
          return await handleCalendarCommand(parsed, context);

        case 'report':
          return await handleReportCommand(parsed, context);

        case 'research':
          return await handleResearchCommand(parsed, context);

        case 'client':
          return await handleClientCommand(parsed, context);

        case 'approve':
          return await handleApproveCommand(parsed, context);

        case 'revise':
          return await handleReviseCommand(parsed, context);

        case 'schedule':
          return await handleScheduleCommand(parsed, context);

        case 'status':
          return await handleStatusCommand(parsed, context);

        case 'help':
          return await handleHelpCommand(parsed, context);

        default:
          // Try to understand as conversational
          return await handleConversational(parsed, context);
      }
    }
```

Session Context (src/cli/session.ts)

```
    interface SessionContext {
      activeClient: string | null;
      lastJobId: string | null;
      pendingApproval: string | null;
```

```
  conversationHistory: Message[];
  settings: UserSettings;
}

class Session {
  private context: SessionContext;

  async processInput(input: string): Promise<Response> {
    // Parse the input
    const parsed = parseCommand(input);

    // Check for conversational context
    if (this.isFollowUp(parsed)) {
      return await this.handleFollowUp(parsed);
    }

    // Route to appropriate handler
    return await routeCommand(parsed, this.context);
  }

  private isFollowUp(parsed: ParsedCommand): boolean {
    // No slash command = likely follow-up
    if (!parsed.raw.startsWith('/')) {
      return this.context.conversationHistory.length > 0;
    }
    return false;
  }

  private async handleFollowUp(parsed: ParsedCommand):
Promise<Response> {
    // Use LLM to understand intent in context
    const intent = await this.classifyIntent(parsed.raw);

    switch (intent) {
      case 'approve':
        return await handleApproveCommand({ command: 'approve' },
this.context);
      case 'revise':
        return await handleReviseCommand({
          command: 'revise',
          topic: parsed.raw
        }, this.context);
      case 'schedule':
        return await handleScheduleCommand({
          command: 'schedule',
          topic: parsed.raw
        }, this.context);
      default:
        return await handleConversational(parsed, this.context);
    }
  }
}
```

Write Command Handler (src/cli/handlers/write.ts)

```
async function handleWriteCommand(
  parsed: ParsedCommand,
  context: SessionContext
): Promise<Response> {
  // Get client
```

```
      const clientId = parsed.flags.client || context.activeClient;
      if (!clientId) {
        return {
          type: 'prompt',
          message: 'Which client is this for?',
          options: await listClients(),
        };
      }

      // Create job
      const jobId = await routerAgent.createJob({
        clientId,
        type: mapContentType(parsed.contentType),
        platform: parsed.platform,
        topic: parsed.topic,
        instructions: parsed.flags.instructions,
      });

      // Update session
      context.lastJobId = jobId;
      context.pendingApproval = jobId;

      // Wait for completion (with streaming updates)
      return await streamJobProgress(jobId);
    }

    async function streamJobProgress(jobId: string): Promise<Response> {
      return new Promise((resolve) => {
        const checkInterval = setInterval(async () => {
          const job = await stateStore.getJob(jobId);

          // Send progress update
          emitProgress({
            jobId,
            status: job.status,
            stage: getStageLabel(job.status),
          });

          // Check if complete
          if (job.status === 'complete' || job.status ===
'human_review') {
            clearInterval(checkInterval);
            resolve(formatContentResponse(job));
          }

          if (job.status === 'failed') {
            clearInterval(checkInterval);
            resolve(formatErrorResponse(job));
          }
        }, 1000);
      });
    }
```

Complete Command Reference

Content Commands

Command	Description	Example
`/write <description>`	Create content with full pipeline	`/write LinkedIn post about AI`
`/quick <description>`	Fast single-shot content	`/quick tweet about our sale`
`/calendar <timeframe>`	Generate content calendar	`/calendar next week`
`/repurpose <source> to <target>`	Convert content	`/repurpose blog to twitter thread`

Analytics Commands

Command	Description	Example
`/report <type>`	Generate report	`/report monthly`
`/research <topic>`	Research a topic	`/research competitor content`
`/trending`	Get trending topics	`/trending in B2B SaaS`

Client Commands

Command	Description	Example
`/client list`	List all clients	`/client list`
`/client switch <name>`	Change active client	`/client switch acme`
`/client new <name> <industry>`	Create client	`/client new "Acme" SaaS`
`/client voice`	View brand voice	`/client voice`
`/client pillars`	View content pillars	`/client pillars`

Workflow Commands

Command	Description	Example
`/approve`	Approve pending content	`/approve`
`/revise <feedback>`	Request changes	`/revise make it shorter`
`/reject <reason>`	Reject content	`/reject off-brand`
`/schedule <when>`	Schedule content	`/schedule monday 9am`
`/publish`	Publish immediately	`/publish now`

Utility Commands

Command	Description	Example
`/status`	Check job status	`/status`
`/history`	View recent jobs	`/history 10`
`/queue`	View scheduled posts	`/queue`
`/help`	Get help	`/help write`
`/settings`	View/update settings	`/settings timezone EST`

Debugging & Troubleshooting

11. Debug Commands

```
/debug <action> [options]
```

Core Debug Commands:

```
/debug status                    # System health overview
/debug job <job_id>              # Deep inspect a job
/debug agents                    # Agent status and health
/debug logs [count]              # Recent system logs
/debug trace <job_id>            # Full event trace for a job
/debug config                    # Show current configuration
/debug connections               # Test all API connections
```

System Status

```
/debug status
```

Output:

```
 SYSTEM STATUS
═══════════════════════════════

 Redis          Connected (latency: 2ms)
 Supabase       Connected (latency: 45ms)
 Claude API     Active (quota: 89% remaining)
 Twitter API    Rate limited (resets in 12m)
 LinkedIn API   Connected

AGENTS
 ├── ● Router        Running (jobs: 47 today)
 ├── ● Context       Running (avg: 120ms)
 ├── ● Brief         Running (avg: 2.1s)
 ├── ● Draft         Running (avg: 4.3s)
 ├── ● Polish        Running (avg: 1.8s)
 └── ● Review        Running (avg: 1.2s)

QUEUES
 ├── Pending jobs:     3
 ├── Processing:       1
 ├── Failed (24h):     2
 └── Completed (24h):  44

Last error: 2h ago (rate limit on Twitter)
```

Job Inspection

```
/debug job <job_id>
```

Output:

```
 JOB INSPECTION: job_abc123
═══════════════════════════════

METADATA
```

```
├── ID:          job_abc123
├── Client:      acme
├── Type:        linkedin_post
├── Status:      failed
├── Created:     2026-02-07 14:23:01 UTC
├── Updated:     2026-02-07 14:23:45 UTC
├── Duration:    44s
└── Iterations:  2/3

INPUT
{
  "topic": "AI automation benefits",
  "platform": "linkedin",
  "instructions": "include statistics"
}

PIPELINE STATUS
├── □ context_loaded    (took 0.12s)
├── □ brief_created     (took 2.3s)
├── □ draft_created     (took 4.1s)
├── □ polish_done       (took 1.9s)
├── □ review            (FAILED)
└── □ complete          (not reached)

ERROR
{
  "stage": "review",
  "error": "API timeout after 30s",
  "stack": "TimeoutError at ReviewAgent.callLLM..."
}

ACTIONS
• /debug retry job_abc123       Retry from failed stage
• /debug trace job_abc123       See full event trace
• /debug dump job_abc123        Export full job data
```

Event Trace

```
/debug trace <job_id>
```

Output:

```
□ EVENT TRACE: job_abc123
═══════════════════════════════

14:23:01.123  job.created
              └── source: router
              └── payload: {type: "linkedin_post", topic: "AI
automation"}

14:23:01.245  context.loaded
              └── source: context-agent
              └── duration: 122ms
              └── payload: {client: "acme", pillars: 4, voice:
"professional"}

14:23:03.567  brief.ready
              └── source: brief-agent
              └── duration: 2322ms
```

```
               └── payload: {wordCount: 150, sections: 3}

14:23:07.891  draft.ready
               └── source: draft-agent
               └── duration: 4324ms
               └── payload: {wordCount: 147, iteration: 1}

14:23:09.234  polish.done
               └── source: polish-agent
               └── duration: 1343ms

14:23:09.456  → review started
               └── source: review-agent

14:23:39.456  □ agent.error
               └── source: review-agent
               └── error: "TimeoutError: API call exceeded 30s"
               └── stack: [expandable]

Total duration: 38.3s
Events: 7
Errors: 1
```

Agent Health

```
/debug agents
```

Output:

```
□ AGENT HEALTH
═══════════════════════════════

ROUTER
├── Status:       □ Running
├── Jobs today:   47
├── Avg latency:  45ms
└── Errors (24h): 0

CONTEXT-AGENT
├── Status:       □ Running
├── Calls today:  47
├── Avg latency:  120ms
├── Cache hits:   89%
└── Errors (24h): 0

BRIEF-AGENT
├── Status:       □ Running
├── Calls today:  47
├── Avg latency:  2.1s
├── Tokens used:  45,230
└── Errors (24h): 1

DRAFT-AGENT
├── Status:       □ Running
├── Calls today:  52 (includes revisions)
├── Avg latency:  4.3s
├── Tokens used:  124,500
└── Errors (24h): 0
```

```
POLISH-AGENT
├── Status:       □ Running
├── Calls today:  48
├── Avg latency:  1.8s
├── Tokens used:  38,900
└── Errors (24h): 0

REVIEW-AGENT
├── Status:       □ Degraded (1 timeout)
├── Calls today:  52
├── Avg latency:  1.2s
├── Pass rate:    87%
├── Tokens used:  28,400
└── Errors (24h): 2

ACTIONS
• /debug agent restart <name>   Restart an agent
• /debug agent logs <name>      View agent-specific logs
```

Logs

```
/debug logs [count] [--level <level>] [--agent <name>]
```

Examples:

```
/debug logs                      # Last 20 logs
/debug logs 50                   # Last 50 logs
/debug logs --level error        # Only errors
/debug logs --agent draft-agent  # Only from draft agent
/debug logs 100 --level warn     # Last 100 warnings
```

Output:

```
□ SYSTEM LOGS (last 20)
```

```
14:45:23 INFO  [router] Job created: job_def456
14:45:23 DEBUG [context-agent] Loading client: acme
14:45:23 DEBUG [context-agent] Cache hit for brand voice
14:45:23 INFO  [context-agent] Context loaded in 89ms
14:45:25 INFO  [brief-agent] Brief created: 145 words
14:45:29 INFO  [draft-agent] Draft created: 142 words
14:45:31 INFO  [polish-agent] Polish complete
14:45:32 INFO  [review-agent] Review passed: 92/100
14:45:32 INFO  [router] Job complete: job_def456

14:23:39 ERROR [review-agent] TimeoutError: API call exceeded 30s
         └── job_id: job_abc123
         └── iteration: 2
         └── stack: TimeoutError at ReviewAgent.callLLM
                    at processEvent (review-agent.ts:45)
                    at MessageBus.consume (message-bus.ts:89)

[... more logs ...]

Showing 20 of 1,247 logs today
Filter: /debug logs --level <debug|info|warn|error>
```

Connection Testing

```
/debug connections
```

Output:

```
 CONNECTION TEST
═══════════════════════════════

Testing Redis...
├── Host:     localhost:6379
├── Ping:      2ms
├── Streams:   Active (3 consumers)
└── Memory:   45MB / 256MB

Testing Supabase...
├── Host:     xxx.supabase.co
├── Ping:      45ms
├── Auth:      Valid
├── Tables:    Accessible (jobs, clients, feedback)
└── Storage:   Connected

Testing Claude API...
├── Endpoint: api.anthropic.com
├── Ping:      120ms
├── Auth:      Valid (sk-ant...XXX)
├── Quota:    89% remaining
└── Model:    claude-sonnet-4-20250514 ✓

Testing Twitter API...
├── Endpoint: api.twitter.com
├── Auth:      Valid
├── Rate:      Limited (resets 14:55)
└── Quota:    12/300 remaining

Testing LinkedIn API...
├── Endpoint: api.linkedin.com
├── Auth:      Valid
└── Quota:    450/500 remaining

All critical connections OK 
1 warning: Twitter rate limited
```

Configuration

```
/debug config
```

Output:

```
 CONFIGURATION
═══════════════════════════════

ENVIRONMENT
├── Node env:     production
├── Version:      1.0.0
├── Started:      2026-02-07 08:00:00 UTC
└── Uptime:       6h 45m

AGENTS
```

```
├── Model:        claude-sonnet-4-20250514
├── Max tokens:   4096
├── Temperature:  0.7 (draft), 0.5 (polish), 0.3 (review)
└── Timeout:      30s

LIMITS
├── Max iterations:    3
├── Max concurrent:    10
├── Rate limit:        100 jobs/hour
└── Max job age:       24h

CLIENTS
├── Default client:    acme
├── Active clients:    5
└── Total clients:     12

STORAGE
├── Redis URL:         localhost:6379
├── Supabase project:  xxx
└── Local memory:      /app/memory/clients/

SENSITIVE VALUES HIDDEN
Use /debug config --show-secrets to reveal (admin only)
```

Recovery Actions

```
/debug retry <job_id> [--from <stage>]     # Retry a failed job
/debug cancel <job_id>                      # Cancel a stuck job
/debug reset <job_id>                       # Reset job to initial
state
/debug flush queue                          # Clear pending jobs
(careful!)
/debug agent restart <name>                 # Restart specific agent
/debug agent restart all                    # Restart all agents
```

Examples:

```
/debug retry job_abc123                     # Retry from failed
stage
/debug retry job_abc123 --from draft        # Retry from draft stage
/debug cancel job_xyz789                    # Cancel stuck job
/debug agent restart review-agent           # Restart review agent
```

Output:

```
 RETRYING JOB: job_abc123
──────────────────────────

Original failure: review (TimeoutError)
Retrying from: review stage
Previous iterations: 2
New iteration: 3

 Sending to review-agent...
 Review passed: 88/100
 Job complete!

Job recovered successfully.
Output: /show job_abc123
```

Data Export

```
/debug dump <job_id> [--format <json|yaml>]   # Export job data
/debug dump logs [--since <time>]             # Export logs
/debug dump client <client_id>                # Export client data
```

Examples:

```
/debug dump job_abc123                        # Export as JSON
/debug dump job_abc123 --format yaml          # Export as YAML
/debug dump logs --since "1 hour ago"         # Export recent logs
/debug dump client acme                       # Export client
profile
```

Interactive Debug Mode

```
/debug shell
```

Enters an interactive debugging shell:

```
 DEBUG SHELL (type 'exit' to leave)
```

```
debug> status
[shows system status]

debug> job job_abc123
[shows job details]

debug> redis KEYS agent:*
1) "agent:router:stats"
2) "agent:brief:stats"
...

debug> sql SELECT * FROM jobs WHERE status = 'failed' LIMIT 5
[runs Supabase query]

debug> eval stateStore.getJob('job_abc123')
[executes code in context]

debug> restart review-agent
Restarting review-agent... done.

debug> exit
Exiting debug shell.
```

Debug Command Reference

Command	Description
`/debug status`	System health overview
`/debug job <id>`	Inspect specific job
`/debug trace <id>`	Full event trace
`/debug agents`	All agent health
`/debug agent <name>`	Specific agent details
`/debug logs [n]`	Recent logs

`/debug config`	Show configuration
`/debug connections`	Test all connections
`/debug retry <id>`	Retry failed job
`/debug cancel <id>`	Cancel stuck job
`/debug reset <id>`	Reset job state
`/debug agent restart <name>`	Restart agent
`/debug dump <target>`	Export data
`/debug shell`	Interactive debug mode

Debug Flags (Global)

Add to any command for extra debugging info:

```
--verbose          # Extra output
--timing           # Show timing breakdown
--raw              # Show raw API responses
--dry-run          # Simulate without executing
```

Examples:

```
/write LinkedIn post about AI --verbose --timing
/debug job job_abc123 --raw
/schedule monday 9am --dry-run
```

Error Handling

Missing Information

```
User: /write a post

Agent: I'd love to help! A few quick questions:

1. What platform? (LinkedIn, Twitter, Instagram, etc.)
2. What topic should the post cover?

Or just tell me naturally: "LinkedIn post about our new feature"
```

Invalid Command

```
User: /postify trending content

Agent: I don't recognize `/postify`. Did you mean:

• `/write` — Create content
• `/quick` — Quick content
• `/research` — Research topics

Type `/help` for all commands.
```

Processing Error

```
User: /report monthly
```

```
Agent: □ Couldn't generate the report.

Issue: LinkedIn API returned an error (rate limited)

Options:
1. Try again in 15 minutes
2. Generate report without LinkedIn data: `/report monthly --skip
linkedin`
3. Check API status: `/status apis`
```

File Structure for CLI Module

```
src/cli/
├── index.ts              # Main entry point
├── parser.ts             # Command parsing
├── router.ts             # Command routing
├── session.ts            # Session management
├── formatter.ts          # Response formatting
├── handlers/
│   ├── write.ts          # /write command
│   ├── quick.ts          # /quick command
│   ├── calendar.ts       # /calendar command
│   ├── report.ts         # /report command
│   ├── research.ts       # /research command
│   ├── client.ts         # /client commands
│   ├── approval.ts       # /approve, /revise, /reject
│   ├── schedule.ts       # /schedule, /publish, /queue
│   ├── status.ts         # /status, /history
│   └── help.ts           # /help, /settings
└── utils/
    ├── nlp.ts            # Natural language parsing
    ├── streaming.ts      # Progress streaming
    └── prompts.ts        # Interactive prompts
```

Copy this entire document into Claude Code and say: "Add this CLI interface to the Luniero implementation. Update the project structure and implement the parser, router, and handlers."